

Clew / Custos 프로젝트

전체 작업 종합 기록 · 지식 정리 · 진행 현황

멀티에이전트 AI 관측가능성(observability) — 토큰 낭비 탐지

필드 테스트 세션 종합본 · 2026

목차

목차	2
0. 이 문서에 대하여	4
1. 한눈에 보기	4
2. Clew가 무엇이고 왜 만드는가	4
쉬운 설명	4
문제와 해결	5
이름의 역사	5
3. 어떻게 탐지하는가 — 캐스케이드 아키텍처	5
쉬운 설명	5
핵심 파라미터와 패턴	6
4. 정직한 검증 철학 (anti-self-deception)	7
쉬운 설명	7
v1의 실패와 피벗	8
5. 합성 검증 결과 (Stage 1 · 2)	8
데이터셋과 동결	8
보정과 단일 평가	9
이 GO가 검증한 것과 아닌 것 (중요)	9
6. 이번 세션의 핵심 — 필드 테스트 (실제 LangGraph)	10
6.1 왜 필드 테스트인가 — 쉬운 설명	10
6.2 발견 1 — llm span 뭉침, 그리고 collapse	10
6.3 TP 보존 확인 — 진짜 낭비도 여전히 잡히나?	11
6.4 발견 2 — 라우터 거짓 양성	11
6.5 발견 3 — 코사인 프로브: JSON 부풀림과 ϕ -transfer	12
6.6 깊은 재해석 — '낭비 중복'은 칼로 자르듯 나뉘지 않는다	13
7. 지금 어디까지 왔나 — 정직한 주장 경계	14
8. 다음 방향	14
9. 관련 논문 종합	16
9.1 MAST — 멀티에이전트 실패 분류 (v1이 사용한 데이터)	16
9.2 IBM — 에이전트 사이클 탐지 (캐스케이드 근거)	16
9.3 IBM — 멀티에이전트 무성 실패 탐지 (인접 연구)	16
9.4 한눈 정리	17
10. 용어집 (쉬운 설명)	17

부록. 동결 파라미터 · 빠른 참조	18
---------------------------	----

0. 이 문서에 대하여

이 문서는 지금까지 Clew(회사명 Custos) 프로젝트에서 함께 진행한 작업을 한 곳에 모아, 당신이 알아야 할 지식과 현재 진행 지점, 그리고 관련 논문을 종합한 기록입니다. 특히 이번 세션의 핵심인 '필드 테스트(실제 LangGraph 트레이스에 탐지기를 붙여본 실험)'의 발견들을 자세히 다룹니다.

각 절은 먼저 '쉬운 설명'으로 개념을 풀고, 그다음 구체적 수치와 결정을 적었습니다. 기술 용어는 9절 용어집에서 다시 풀어 설명합니다. 도표는 모두 흑백입니다.

1. 한눈에 보기

Clew는 여러 AI 에이전트가 협업하는 시스템의 실행 기록(트레이스)을 분석해, '같은 일을 쓸데 없이 반복하며 토큰을 태우는 구간'을 찾아내는 도구입니다. 토큰은 곧 비용이므로, 이 낭비를 잡아주면 기업의 LLM 운영비를 줄일 수 있다는 것이 제품의 약속입니다.

지금까지의 결론을 세 줄로 요약하면 다음과 같습니다.

1. 탐지 '로직'은 합성 데이터에서 깨끗하게 작동함을 단일 평가로 확인했습니다(F1 0.857, 거짓 양성 0).
2. 그 탐지기를 실제 LangGraph 트레이스에 붙이는 '필드 테스트'에서, 합성 검증이 결코 건드리지 못했던 세 가지 현실 문제를 발견했습니다. 둘은 수정 가능한 엔지니어링 문제이고, 하나는 더 깊은 문제(임계값 ϕ 가 실제 분포로 옮겨가는지)입니다.
3. 그래서 다음 방향은 '랩에서 더 깊이 파기'가 아니라 '실제 트레이스와 실제 사용자'로의 이동입니다. 남은 질문들은 모두 현실에서만 답할 수 있기 때문입니다.

2. Clew 가 무엇이고 왜 만드는가

쉬운 설명

여러 명의 직원(에이전트)이 한 프로젝트를 나눠 맡았다고 상상해 보세요. 그런데 A직원이 이미 찾아둔 자료를 B직원이 또 찾고, C직원은 앞사람의 결론을 같은 내용으로 다시 쓰고, 어떤 작업

은 끝났는데도 멈추지 못하고 계속 돕니다. 사람이라면 '그거 아까 했잖아'라고 말해주겠지만, AI 에이전트들은 그걸 모른 채 매번 비싼 '생각 비용(토큰)'을 지불합니다. Clew는 이 '아까 했잖아'를 자동으로 짚어주는 감시자입니다.

문제와 해결

문제: 멀티에이전트 LLM 시스템은 빠르게 늘어나는데 만족도는 낮고, 가장 큰 고통은 '토큰 비용 폭증'입니다. 기존 관측 도구(Langfuse, LangSmith, Arize/Phoenix, Datadog 등)는 지연시간·토큰 '양'은 보여주지만, '구조적 반복'과 '의미적 중복'을 자동으로 잡아주지는 못합니다.

해결: Clew는 트레이스에서 '낭비되는 반복 루프와 핸드오프'를 탐지해 보고합니다. 즉 '실패를 예측한다'가 아니라 '토큰을 태우는 중복을 잘라낸다'가 제품의 이야기입니다.

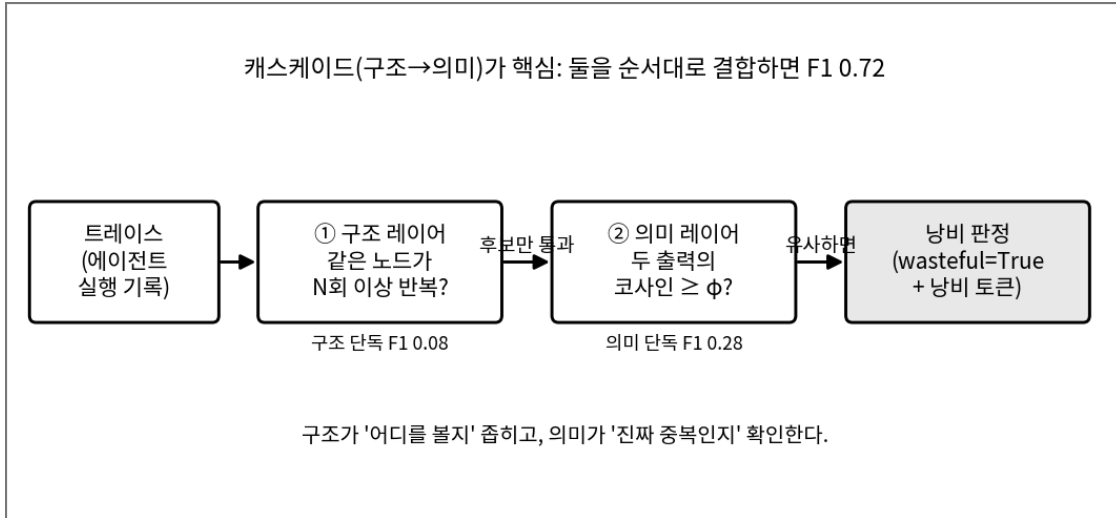
이름의 역사

프로젝트는 EmergenceTrace → Spindown을 거쳐 현재 회사명 Custos, 제품명 Clew로 자리잡았습니다. 모든 개발은 Claude Code CLI로 진행합니다.

3. 어떻게 탐지하는가 — 캐스케이드 아키텍처

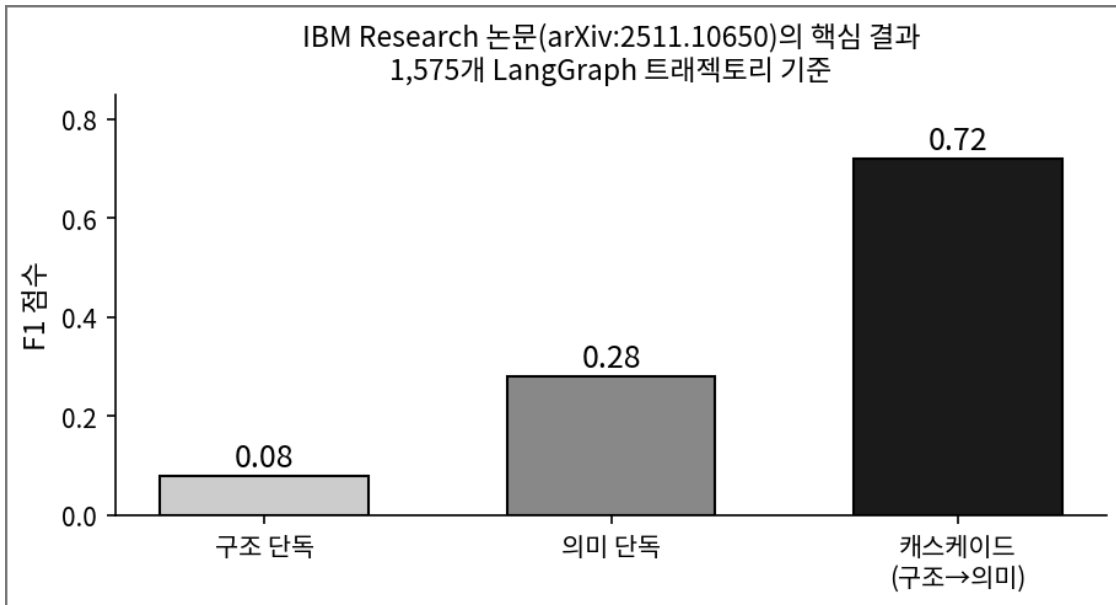
쉬운 설명

낭비를 찾는 데는 두 가지 눈이 필요합니다. 첫째 '구조의 눈'은 '같은 노드가 여러 번 등장했나?'처럼 모양을 봅니다. 둘째 '의미의 눈'은 '두 출력이 사실상 같은 내용인가?'처럼 뜻을 봅니다. 핵심 통찰은 이 둘을 '순서대로' 결합하는 것입니다. 구조의 눈이 '여기 수상하다'고 좁혀주면, 의미의 눈이 '정말 중복인지' 확인합니다. 둘 중 하나만 쓰면 약하고, 순서대로 이으면(캐스케이드) 강해집니다.



[그림 1] 캐스케이드 탐지 구조 — 구조 레이어가 후보를 좁히고, 의미 레이어가 진짜 중복인지 확인한다.

이 '구조 단독은 약하고, 의미 단독도 약하지만, 순서대로 이으면 강해진다'는 점은 외부 연구 (IBM Research)에서도 동일하게 관찰됩니다. 아래 그림이 그 수치입니다.



[그림 2] 구조 단독(0.08) · 의미 단독(0.28) · 캐스케이드(0.72) — 결합의 효과가 명확하다. (외부 연구 결과)

핵심 파라미터와 패턴

- 임계값 ϕ (파이) = 0.514345 : 두 출력의 코사인 유사도가 이 값 이상이면 '의미적으로 중복'으로 본다.
- 반복 임계 $N = 2$: 같은 노드가 2 회 이상이면 구조적 반복 후보로 본다.

- 임베딩 모델 : paraphrase-multilingual-MiniLM-L12-v2 (문장을 숫자 벡터로 바꿔 유사도를 재는 다국어 모델).

탐지 대상 낭비 패턴은 네 가지로 정의했습니다.

패턴	뜻	상태
repeat_node	같은 노드가 거의 같은 출력으로 반복	범위 내
pingpong_aba	$A \rightarrow B \rightarrow A$ 식으로 두 노드가 핑퐁	범위 내
requery_known	이미 가진 정보를 같은 도구로 재조회	범위 내
regen_handoff	다음 에이전트가 앞 결과를 동일 내용으로 재생성	범위 제외(아래 설명)

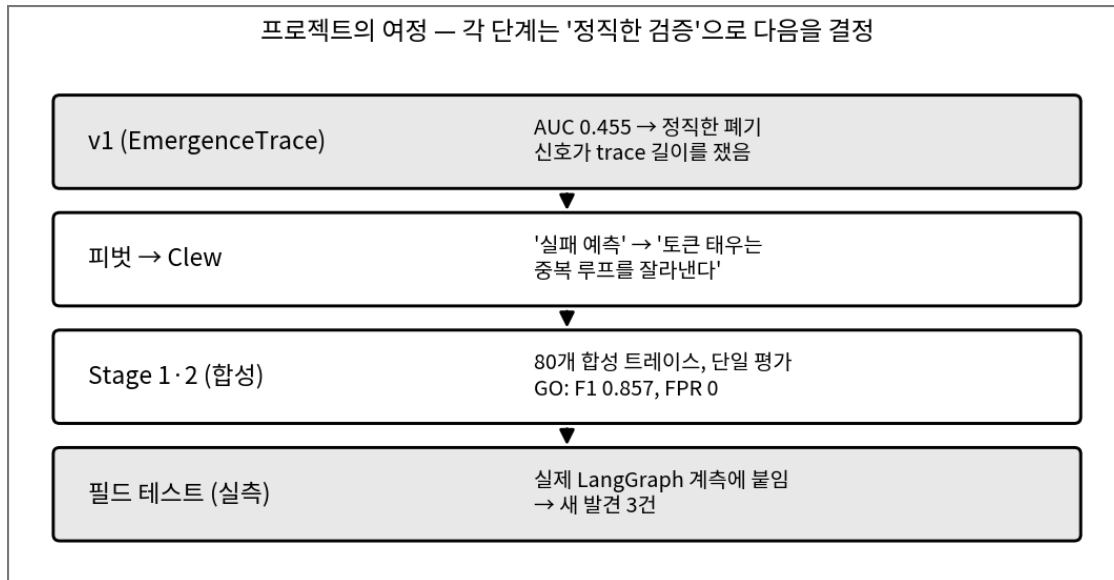
requery_known은 '입력 게이트'라는 장치로 처리합니다. 도구(tool) 호출의 반복 후보는 '입력이 같은 경우'에만 잡도록 해서, 같은 양식이지만 값이 다른 조회(예: 고객 A 조회 vs 고객 B 조회)를 중복으로 오해하지 않게 만들었습니다.

regen_handoff 범위 제외 이유: 이 패턴은 서로 다른 노드($A \rightarrow B$) 사이의 의미적 재생성이라, 강한 구조 신호가 없어 의미 레이어에만 의존하게 됩니다. 그러면 거짓 양성 위험이 커지므로, 정직하게 v1 범위에서 뺐습니다. '할 수 있는 척' 대신 '아직 못 한다'를 택한 것입니다.

4. 정직한 검증 철학 (anti-self-deception)

쉬운 설명

이 프로젝트의 가장 중요한 원칙은 '스스로를 속이지 않기'입니다. 결과를 보기 전에 성공/실패 기준을 미리 적어두고, 결과가 나오면 사후에 기준을 조정하지 않습니다. 빨간불이 켜지면 멈춥니다. 평가용 데이터는 단 한 번만 만집니다. 이렇게 하는 이유는 v1에서 빠져린 교훈을 얻었기 때문입니다.



[그림 3] 프로젝트의 여정 — 각 단계는 '정직한 검증'으로 다음을 결정해 왔다.

v1의 실패와 피벗

초기 버전(EmergenceTrace)은 'Loop Pressure'와 'Information Gain Decay'라는 신호로 실패를 예측하려 했습니다. 사전 등록한 성공 기준은 $AUC \geq 0.80$ 이었는데, 실제 결과는 $AUC \approx 0.455$ 로 무작위보다도 못했습니다. 분석해 보니 그 신호는 사실 '실패'가 아니라 '트레이스 길이'를 재고 있었습니다. 이것을 숨기지 않고 정직하게 폐기(kill)한 뒤, '토큰을 태우는 중복 루프를 잘라낸다'는 더 좁고 단단한 썰기로 피벗했습니다.

이 철학을 구현한 안전장치가 '3중 누수 가드'입니다. (1) 코드 정적 분석, (2) 런타임 탐침, (3) 탐지 모듈 비어있음 단언 — 탐지 코드가 정답 라벨 파일을 읽거나 불러오지 못하도록 세 겹으로 막아, '답을 몰래 보고 맞히는' 자기기만을 원천 차단합니다.

5. 합성 검증 결과 (Stage 1 · 2)

데이터셋과 동결

80개의 짝지은 라벨 트레이스(낭비 40 / 깨끗함 40)를 네 패턴에 걸쳐 만들고 git으로 동결했습니다. 보정(calibration)은 오직 개발 셋(seed=7)에서만 하고, 평가 셋(seed=42)은 파라미터를 동결한 뒤 단 한 번만 실행한다는 규율을 지켰습니다.

보정과 단일 평가

개발 셋에서 사전 등록한 기준 C1-C4를 모두 통과했습니다(분포 간격 +0.2208, Cohen's d 4.38, 거짓 양성률 0). 동결 직전 재현율 점검에서 범위 내 세 패턴은 30/30으로 완벽했고, 범위 밖 regen_handoff는 0/10이었습니다(예상된 결과).

파라미터를 동결한 뒤 평가 셋(seed=42)을 단 한 번 실행한 결과:

지표	값
판정	GO
F1	0.8571
거짓 양성률 (FPR)	0.0000
TP / FP / TN / FN	30 / 0 / 40 / 10
범위 내 재현율 (TPR)	1.0 (30/30)

동결 태그: stage1-freeze, stage2-detector-freeze, stage2-eval-freeze, stage2-eval-go.

이 GO가 검증한 것과 아닌 것 (중요)

검증한 것: 사전 등록한 '합성' 데이터에서, 세 패턴에 대해 탐지 '로직'이 심은 중복을 잡고 깨끗한 것은 거짓 양성 0으로 통과시킨다.

검증하지 못한 것: 실제 프로덕션 트래이스, 실제 토큰 절감액, 실제 수요. 따라서 정직한 공개 표현은 '실제로 토큰을 아껴준다'가 아니라 '합성 트래이스에서 심은 세 가지 낭비 패턴을 거짓 양성 0으로 잡는다'입니다.

6. 이번 세션의 핵심 — 필드 테스트 (실제 LangGraph)

6.1 왜 필드 테스트인가 — 쉬운 설명

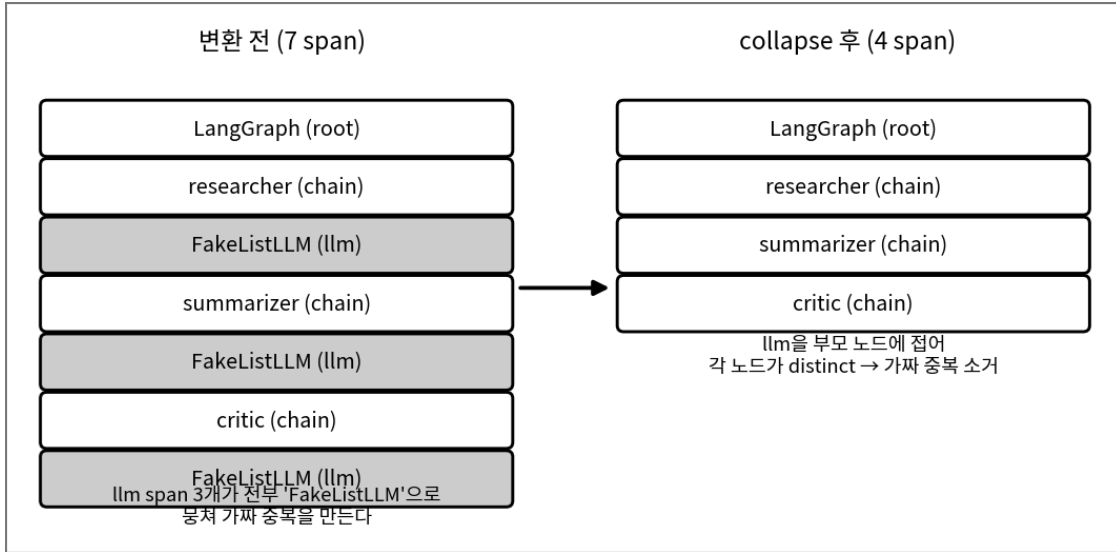
합성 평가는 '우리가 직접 만든 깨끗한 트레이스'로 탐지기를 시험했습니다. 하지만 합성 평가는 '어댑터'(실제 트레이스를 우리 형식으로 바꾸는 변환기)를 한 번도 거치지 않았습니다. 그래서 '실제 LangGraph 에이전트를 돌려서 나온 진짜 트레이스'에 붙여보는 것이 다음 관문이었습니다. 비유하자면, 실험실에서 잘 작동한 기계를 처음으로 '진짜 도로'에 올려본 것입니다.

이 단계에서 합성 검증이 결코 보지 못한 세 가지가 드러났습니다. 이것이 바로 '정직한 검증'이 계속 일하고 있다는 증거입니다.

6.2 발견 1 — llm span 뭉침, 그리고 collapse

실제 LangGraph 트레이스를 OpenInference로 계측하면, 각 노드(researcher/summarizer/critic)는 잘 구분됩니다. 그런데 그 안의 'LLM 호출' span 3개가 전부 모델 클래스 이름(FakeListLLM) 하나로 '뭉쳐서' 들어왔습니다. 탐지기는 이를 '같은 노드가 3번 반복'으로 오해해 가짜 중복(거짓 양성)을 만들었습니다. 탐지기 로직의 잘못이 아니라, 어댑터가 실제 계측 트리를 '논리 노드 단위'로 맞추지 못한 'identity 갭'이었습니다.

해결책은 collapse_to_logical_nodes입니다. LLM 하위 span은 부모 노드에 '접고', 도구(tool) span은 그대로 두며(요청 재조회 탐지에 필요), 그래프 루트는 한 번만 등장하므로 자연스럽게 제외됩니다. 변환 후 노드 id가 모두 구분되어 가짜 중복이 사라졌습니다.



[그림 4] collapse 전후 — 뭉쳐 있던 llm span 3개를 부모 노드에 접어 각 노드를 distinct하게 만든다.

결과: 깨끗한 실제 트레이스에서 거짓 양성이 처음으로 0이 되었습니다(단, 이때는 '진짜 낭비가 없는' 트레이스였습니다).

6.3 TP 보존 확인 — 진짜 낭비도 여전히 잡히나?

거짓 양성을 없애는 변환이 '진짜 탐지'까지 죽이면 안 됩니다. 그래서 repeat_node를 일부러 심은 트레이스(같은 researcher 노드를 패러프레이즈로 두 번 실행)를 만들어, collapse 후에도 탐지기가 발화하는지 확인했습니다. 결과는 코사인 0.927로 정확히 FIRE — TP(참 양성)가 보존되었습니다.

주의할 경계: 이 트레이스는 깨끗한 흐름이라 이 시험이 확인한 것은 '실제 계측에서 repeat_node 한 패턴의 탐지가 작동한다'는 기계적 사실이지, 통계적 재현율(여러 패턴·대량 표본)이 아닙니다.

6.4 발견 2 — 라우터 거짓 양성

위 시험에서 예상 밖의 구조 후보가 하나 더 나왔습니다. LangGraph의 '조건부 엣지 함수'(should_loop, 다음에 어디로 갈지 정하는 라우터)도 chain span으로 계측되어, 반복되면 '노드처럼' 중복 후보가 된 것입니다. 그때는 라우터의 두 출력이 우연히 달라서 발화하지 않았을 뿐이었습니다.

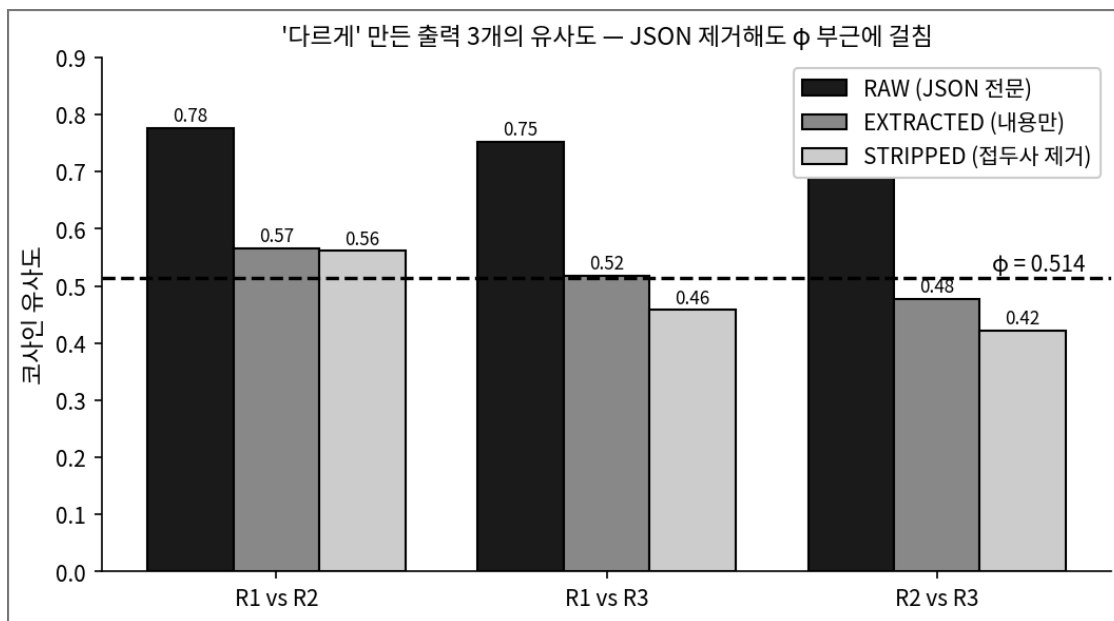
이를 '우연'으로 넘기지 않고, 3회 이상 도는 루프에서 라우터가 같은 값을 반복 반환하는 적대적 케이스를 만들어 '진짜 거짓 양성'을 재현했습니다(코사인 1.0으로 FIRE). 버그가 실재함을 먼저 보여준 뒤(=회귀 테스트 확보), 구별 신호를 조사했습니다.

핵심 원칙: Clew의 주장은 '토큰 낭비'다. 그렇다면 탐지기는 '토큰을 태우는 span'만 보면 된다. 라우터처럼 LLM/도구를 안 거치는 span은 토큰을 0 태우므로 정의상 토큰 낭비가 아니다 → 제외하는 것이 맞다. 신호 `has_llm_or_tool_child`(자식에 `llm/tool`이 있는가)는 이 정의를 그대로 구현한 것이며, 프레임워크에 독립적이다.

6.5 발견 3 — 코사인 프로브: JSON 부풀림과 ϕ -transfer

가장 무거운 발견입니다. 위 시험에서 '일부러 다르게' 만든 researcher 출력 3개(서로 다른 주제)가 코사인 0.70–0.78로 전부 ϕ 를 넘어 '중복'으로 오판되었습니다. 원인을 수치로 분리하기 위해, 같은 텍스트를 세 가지 표현으로 비교했습니다.

- RAW : 탐지기가 실제로 임베딩하던 그대로(JSON 상태덕 전문).
- EXTRACTED : JSON 에서 실제 내용(research 필드)만 추출.
- STRIPPED : 거기서 공유 접두사('N 번째 조사 결과:')까지 제거.



[그림 5] 핵심 도표 — JSON을 벗겨도(EXTRACTED) 유사도가 $\phi(0.514)$ 부근에 걸친다. ϕ 초과 쌍: RAW 3/3 → EXTRACTED 2/3 → STRIPPED 1/3.

판정은 두 원인이 '둘 다'였습니다.

(a) **확정 — JSON 부풀림:** 스케폴드를 제거하니 0.70–0.78 → 0.48–0.57로 약 0.2pt 하락했습니다. 즉 어댑터가 output.value의 내용을 추출해 임베딩하도록 고쳐야 합니다(수정 가능한 엔지니어링 문제).

(b) **잔존 — ϕ -transfer 문제:** 내용만 추출해도 EXTRACTED 밴드가 0.48–0.57에 앉아 $\phi=0.514$ 가 그 한가운데를 가릅니다. 합성으로 보정한 임계값이 실제 출력 분포로 깨끗하게 옮겨가지 않는다는 신호입니다.

6.6 깊은 재해석 — '낭비 중복'은 칼로 자르듯 나뉘지 않는다

실제 출력에서 '낭비되는 중복'은 0/1로 딱 떨어지는 개념이 아닙니다. 같은 도메인의 진짜 출력들은 자연스럽게 '중간 정도'의 유사도 대역에 모이고, 고정된 임계값 ϕ 하나는 그 중간을 늘 일부 오분류합니다. 합성 데이터는 '심은 거의-동일 vs 무관한 깨끗함'으로 이 경계를 인위적으로 또렷하게 만들었기에 임계값이 잘 먹혔던 것입니다. 현실은 가운데가 짝 차 있습니다.

이는 v1의 실패와 같은 계열의 교훈입니다. 그때는 신호가 'trace 길이'를 켜고, 이번에는 임계값의 '외적 타당성'이 합성에만 묶여 있었습니다. 다만 표본이 $n=3$ 이므로, 이것은 '강하게 의심해야 할 신호'이지 '확정된 결론'은 아닙니다. 진짜 답은 실제 출력 분포로만 나옵니다.

7. 지금 어디까지 왔나 — 정직한 주장 경계

필드 테스트가 끝난 현재, '검증된 것'과 '아직 못 한 것'을 한 장에 정리하면 다음과 같습니다.

지금 정직하게 주장할 수 있는 것 vs 아직 못 하는 것	
체크 = 검증됨 / 삼각형 = 보완 가능 / 엑스 = 미검증	
✓	합성 데이터에서 탐지 로직 구조→의미 캐스케이드가 심은 중복을 잡고 clean은 통과 (단일 평가 GO)
✓	실제 계측에서 구조적 기제 어댑터 변환 · collapse가 llm 뭉침 FP 제거 · TP(반복 노드) 보존
△	라우터 FP (수정 가능) conditional-edge가 노드로 오인됨 → has_llm_or_tool_child로 제외 예정
△	JSON 임베딩 부풀림 (수정 가능) output.value 내용만 추출하면 유사도 약 0.2pt 하락
✕	φ의 실측 transfer 실제 같은-도메인 출력이 φ 부근에 몰림 → 미검증, 실데이터 재보정 필요
✕	실제 수요 · 토큰 절감 실측 실제 에이전트가 잡을 낭비를 만드는가 / 돈 낼 통증인가 — 미검증

[그림 6] 정직한 주장 경계 — 구조적 기제는 실측에서 확인, 그러나 φ의 실측 transfer와 실제 수요는 미검증.

공개 글에서 정직한 현재 표현: '실제 계측에서 구조적 기제(어댑터·collapse·TP 보존)는 확인했다. 다만 의미 임계값(φ)이 실제 출력 분포로 옮겨가는지는 미검증이며, 실데이터 재보정이 필요하다.' — '실측에서 작동한다'는 아직 주장할 수 없습니다.

하지 말아야 할 유혹: EXTRACTED의 두 초과 쌍이 0.52·0.57로 φ 바로 위라, 'φ를 조금만 올리면 된다'는 생각이 들 수 있습니다. 그러나 예시 몇 개에 맞춰 φ를 손으로 올리는 것은 정확히 v1의 사후 오버핏입니다. φ는 실제 분포에서 정직하게 다시 잡아야 합니다.

8. 다음 방향

필드 테스트의 진짜 선물은 버그 목록이 아니라 '이제 랩에서 답할 수 있는 것은 거의 다 나왔다'는 신호입니다. 남은 1차 질문(φ가 실측에 transfer되나, 통증은 실재하나, 돈 낼 만한가, 고정 임계값이 맞는 제품 형태인가)은 모두 '현실과의 접촉'에서만 답이 나옵니다. 권고하는 순서는

다음과 같습니다.

4. 싼 어댑터 수정 둘을 끝내고 승격(시간 제한). 라우터 제외(has_llm_or_tool_child)와 JSON 내용 추출 — 둘 다 의심 없고 저렴합니다. collapse 와 함께 실제 어댑터로 올리고 필드 테스트들을 회귀로 재실행합니다. 단 '여기까지만' — 부풀리지 않습니다.
5. (선택) 실측 출력 코퍼스 숫자 하나. 실제 LLM 로 다양한 과제를 돌려 '과제 간(=비중복) 쌍'의 코사인 분포를 보고, ϕ 에서의 경험적 거짓 양성률을 쟀다. 목적은 'finding 3 이 n=3 우연이 아니라 스케일에서도 진짜인지' 확인 + 대화용 숫자입니다. 끝없는 재보정 프로젝트로 만들지 않습니다.
6. (진짜 방향) 준비해둔 커뮤니티 아웃리치를 '발표'가 아니라 '트레이스+수요 발굴'로 용도 변경. 실제 멀티에이전트 시스템을 돌리는 3-5 명을 찾아 트레이스를 받고(= ϕ 를 정직하게 재보정할 유일한 길), 그 통증이 실재하는지 듣는다(=수요 검증). 실제 트레이스는 이 둘을 동시에 해결합니다.

새로운 자기기만 경계: 탐지기를 랩에서 완벽하게 다듬는 것 자체가, 더 무섭지만 더 중요한 '수요' 질문을 피하는 방법이 될 수 있습니다. 코퍼스를 또 만들고 ϕ 를 또 재보정하는 것은 '모르는 사람에게 트레이스를 보여달라'고 말하는 것보다 훨씬 편안합니다. 그 편안함이 지금의 함정입니다. 한 발은 랩 마무리(1), 무게중심은 현실(3)에 둡니다.

9. 관련 논문 종합

이 프로젝트의 근거가 된 외부 연구를 정리합니다. 핵심 원칙: 이들은 '외부 연구'이지 Clew 자신의 검증 결과가 아닙니다. 공개 글에서 이 구분을 반드시 유지하세요.

9.1 MAST — 멀티에이전트 실패 분류 (v1 이 사용한 데이터)

Cemri et al., UC Berkeley, arXiv:2503.13657. 7개 프레임워크에서 수집한 주식 트레이스 데이터셋. v1 실험이 이 데이터를 사용했습니다.

▲ **건수 확인 필요:** 원논문/요약은 약 1,642건('1,600+')으로 표기합니다. 이전 기록의 1,242건이 (a) v1에서 실제 처리한 부분집합인지 (b) 전사 오류인지, v1 실험 산출물의 실제 로드 건수로 확인한 뒤 공개 글 숫자를 통일하세요. AUC 결과 자체는 어느 쪽이든 불변입니다(서술 오류이지 결과 오류가 아님).

9.2 IBM — 에이전트 사이클 탐지 (캐스케이드 근거)

George, Kumar, Pathak, Ray, Verma, Moogi (IBM Research), "Unsupervised Cycle Detection in Agentic Applications," arXiv:2511.10650. 구조(콜스택)+의미(유사도) 결합 프레임워크. F1 0.72(precision 0.62, recall 0.86), 구조 단독 0.08, 의미 단독 0.28.

- IBM Research 귀속 확인됨: 저자 Kaustabha Ray·Pratibha Moogi 등이 IBM Research 소속(dblp/ACM 확인).
- 평가는 IBM 데이터가 아님: 1,575 개 trajectory(LangGraph 기반 주식 애플리케이션, 저자들 자체 큐레이션 벤치마크, 논문 채택 후 공개 예정). 공개 글에선 '1,575 개 LangGraph 트래젝토리에서 측정'으로 정확히 기술.

이 논문은 Clew의 '구조→의미 캐스케이드' 아키텍처를 뒷받침하지만, Clew의 자체 검증(합성 GO)과는 별개라는 점을 유지하세요.

9.3 IBM — 멀티에이전트 무성 실패 탐지 (인접 연구)

Pathak et al. (IBM), "Detecting Silent Failures in Multi-Agent AI Trajectories," arXiv:2511.04032. 사용자 행동·비결정성·LLM 변동성을 담은 4,275 및 894개 trajectory 벤치마크

큐레이션, 이상 탐지 벤치마킹(지도 XGBoost·준지도 SVDD가 최대 98%·96% 정확도). Clew와 같은 저자군이 인접 문제를 다룬다는 맥락 정보.

9.4 한눈 정리

논문	무엇	Clew에의 의미
MAST (2503.13657)	멀티에이전트 실패 분류 데이터 (~1,642건)	v1 실험의 데이터 출처
IBM Cycle (2511.10650)	구조+의미 캐스케이드, F1 0.72	아키텍처의 외부 근거
IBM Silent (2511.04032)	무성 실패 이상 탐지 벤치마크	인접 연구 맥락

10. 용어집 (쉬운 설명)

용어	뜻
트레이스(trace)	에이전트가 실행되며 남기는 단계별 기록. 누가 무엇을 언제 했는지의 로그.
span	트레이스 안의 한 조각(한 노드 실행, 한 LLM 호출, 한 도구 호출 등).
노드(node)	LangGraph에서 하나의 작업 단위(예: researcher, summarizer).
OTel / OpenInference	에이전트 실행을 표준 형식으로 기록하는 계측 규격.
어댑터(adapter)	실제 계측 트레이스를 Clew 내부 형식으로 바꾸는 변환기.
임베딩(embedding)	문장을 의미를 담은 숫자 벡터로 바꾼 것.
코사인 유사도	두 벡터가 얼마나 같은 방향인지(0~1에 가까움). 1에 가까울수록 비슷.
ϕ (파이)	유사도가 이 값을 넘으면 '중복'으로 보는 임계값(0.514345).
캐스케이드	구조 레이어로 후보를 좁힌 뒤 의미 레이어로 확인하는 2단 결합.
collapse	llm 하위 span을 부모 노드에 접어 '논리 노드 단위'로 만드는 변환.
FP / TP	FP=거짓 양성(아닌데 낭비라고 함), TP=참 양성(진짜 낭비를 잡음).
FPR / TPR	FPR=거짓 양성률(정밀도 관련), TPR=재현율(놓치지 않는 정도).
F1	정밀도와 재현율의 조화평균. 탐지 품질의 종합 점수.
누수 가드	탐지 코드가 정답 라벨을 몰래 보지 못하게 막는 안전장치.

ϕ -transfer	합성으로 정한 ϕ 가 실제 데이터 분포에서도 통하는지의 문제.
------------------	---

부록. 동결 파라미터 · 빠른 참조

항목	값
ϕ (코사인 임계값)	0.514345
N (반복 임계)	2
임베딩 모델	paraphrase-multilingual-MiniLM-L12-v2
검증 데이터셋	80개 짝지은 트레이스 (낭비 40 / 깨끗함 40)
단일 평가(seed=42) 결과	GO, F1 0.8571, FPR 0, TP/FP/TN/FN = 30/0/40/10
동결 태그	stage1-freeze, stage2-detector-freeze, stage2-eval-freeze, stage2-eval-go
주요 외부 근거	George et al.(IBM), arXiv:2511.10650 — 캐스케이드 F1 0.72
개발 환경	Claude Code CLI

이 문서는 현재까지의 작업과 발견을 종합한 것이며, 다음 결정(어댑터 수정 → 실측 코퍼스 → 실제 트레이스/수요 발굴)의 출발점으로 사용하도록 작성되었습니다.